

You can query the values of the *read-only attributes*, but you cannot set them as *Plant Simulation* computes the value for the point-in-time at which you query it. In most cases a *read-only attribute* corresponds to an unavailable dialog item on one of the tabs of the object, for example on the tab **Statistics**.

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window **Show Attributes and Methods**. The figure below illustrates the information using the example of the object *Station*.

Name	Value	Inherited	Watchable	Signature
NumPartsSinceSetup	0			-> integer
NumPred	1			-> integer
NumSucc	3			-> integer

Name of the read-only attribute Current value of the read-only attribute Data type of the return value

- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected [Class \[general description\]](#).
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected [Instance \[general description\]](#).

To query the value of a *read-only attribute*, you might, for example, type:

```
print Source.Empty
```

~ [SimTalk] (tilde) - material flow objects

Returns the object located directly above the object designated by <Path> in the hierarchy of the objects.

Remarks

The location of the *MU* is the object on which it is currently located. All other objects return the *Frame* in which they are located as the location.

Type

Read-only attribute

Syntax

```
<Path>~ -> object
<Path>.Location -> object
```

Watchable

The attribute is [watchable](#).

Return Value

The return value has the data type *object*.

VOID if the *Worker* or the *AGV* are on their way to their destination.

Examples

```
print "shaft is located on:", @.~
print "shaft is located on:", @.Location
```

```
print Preparing~ -- Preparing is the name of a Conveyor
-- returns .Models.MyCarAssembly.PaintShop.PaintLine1
-- PaintLine1 is the name of the Frame
```

```
print .MUs.Part:9~ -- location of Part 9
-- returns .Models.MyCarAssembly.PaintShop.PaintLine1.Preparing
-- Preparing is the name of a Conveyor
```

SimTalk

[LocationInFrame \[SimTalk\]](#)

~ [SimTalk] (tilde) - material flow objects

See also

[Open Location \[described\]](#)



on the [Home Ribbon Tab](#)

Class [SimTalk] - read-only attribute

Returns the class in the *Class Library* from which the object designated by <Path> was derived or instantiated, possibly over several levels.

Remarks

Compare this example: We copied the object *Frame* three times and named it *Frame1*, *Frame2*, and *Frame3*. Then, we insert *Frame1* into *Frame2*, and *Frame2* into *Frame3*. When we call `.Frame3.Frame2.Frame1.Class` the result will be *.Frame1* because *.Frame1* is the object in the *Class Library* from which `.Frame3.Frame2.Frame1` was derived.

If you use *Class* for an instance of an object, *Plant Simulation* moves up in the inheritance structure until it reaches an object in the *Class Library*. It then opens this object.

Type

Read-only attribute

Syntax

```
<Path>.Class → object
```

Return Value

The return value has the data type *object*.

Example

```
if class_change  
  current.Class.openDialog  
end
```

See also

[Class \[general description\]](#)

[Open Class \[described\]](#)

[Go to Class \[described\]](#)

InternalClassType [SimTalk]

Returns the unique built-in **English** object name that describes the type of the object designated by <Path>.

Remarks

Note

The *InternalClassType* is not the localized object name that some of the localized versions of *Plant Simulation* show, but the **English** object name which we assigned.

Return Value	(Information Only
English Object Name	Respective German Object Name)
ActiveX	ActiveX
AngularConverter	Eckumsetzer
AssemblyStation	Montagestation
AttributeExplorer	AttributExplorer
Basis	Basis
Broker	Broker
Buffer	Puffer
Button	Schaltfläche
Chart	Diagramm
Checkbox	Kontrollkästchen
Comment	Kommentar
Connector	Kante
Container	Förderhilfsmittel
Converter	Umsetzer
Conveyor	Förderstrecke
Cycle	Takt
DataList	Kartei
DataStack	Stapel
DataTable	Tabelle
DatatQueue	Warteschlange

Return Value	(Information Only
English Object Name	Respective German Object Name)
DePortioner	Entportionierer
Dialog	Dialog
DismantleStation	Demontagestation
Display	Display
Drain	Senke
DropDownList	Dropdownliste
EventController	Ereignisverwalter
Exporter	Exporter
FileInterface	Dateischnittstelle
FileLink	Dateiverknüpfung
FlowControl	Flusssteuerung
FluidDrain	Flüssigkeitssenke
FluidSource	Flüssigkeitsquelle
Folder	Ordner
FootPath	Fußweg
Frame	Netzwerk
GanttChart	GanttDiagramm
GAOptimization	GAOptimierung
GARangeAllocation	GABereichszuordnung
GASelection	GASelektion
GASequence	GAREihenfolge
GASetAllocation	GAMengenzuordnung
Generator	Generator
Interface	Übergang
LockoutZone	Schutzkreis
Method	Methode
Mixer	Mixer

Return Value	(Information Only
English Object Name	Respective German Object Name)
MQTT	MQTT
ODBC	ODBC
OPCClassicInterface	OPCClassicSchnittstelle
OPCUAInterface	OPCUASchnittstelle
Oracle	Oracle
ParallelStation	Parallelstation
Part	Fördergut
PatchMatrix	PatchMatrix
PickAndPlace	PickAndPlace
Pipe	Rohr
PlaceBuffer	PlatzPuffer
Portioner	Portionierer
Report	Bericht
ShiftCalendar	Schichtkalender
SIMIT	SIMIT-Schnittstelle
Socket	Socket
Sorter	Sortierer
Source	Quelle
SQLite	SQLite
Station	Einzelstation
Store	Lager
Tank	Tank
Teamcenter	Teamcenter
TimeSequence	Zeitleiste
Toolbar	Symbolleiste
Track	Weg
Transporter	Fahrzeug

Return Value	(Information Only
English Object Name	Respective German Object Name)
Trigger	Trigger
Turnplate	Drehplatte
Turntable	Drehtisch
TwoLaneTrack	ZweispurigerWeg
Variable	Variable
Worker	Werker
WorkerPool	WerkerPool
Workplace	Arbeitsplatz
XMLInterface	XMLSchnittstelle

Type

Read-only attribute

Syntax

```
<Path>.InternalClassType → string
```

Return Value

The return value has the data type *string*.

Examples

```
print MyStation.InternalClassType
// returns "Station", works for a Station, which you inserted into a Frame
```

```
print .UserObjects.MyStation.InternalClassType
// returns "Station"
```

SimTalk

[_3D.InternalClassType \[SimTalk\]](#)

[typeof \[SimTalk\]](#)

See also

Show Attributes and Methods [described]

Location [SimTalk] - material flow objects

Returns the object located directly above the object designated by <Path> in the hierarchy of the objects.

Remarks

Instead of *Location* you can also type the tilde ~.

The location of the *MU* is the object on which it is currently located. All other objects return the *Frame* in which they are located as the location.

Type

Read-only attribute

Syntax

```
<Path>.Location -> object  
<Path>~ -> object
```

Watchable

The attribute is **watchable**.

Return Value

The return value has the data type *object*.

VOID if the *Worker* or the *AGV* are on their way to their destination.

Examples

```
print "shaft is located on:", @.Location  
print "shaft is located on:", @.~
```

```
print Preparing.Location -- Preparing is the name of a Conveyor  
-- returns .Models.MyCarAssembly.PaintShop.PaintLine1  
-- PaintLine1 is the name of the Frame
```

```
print Preparing.Location -- Preparing is the name of a Conveyor  
-- returns .Models.MyCarAssembly.PaintShop.PaintLine1  
-- PaintLine1 is the name of the Frame
```

```
print .MUs.Part:9.Location -- location of Part 9  
-- returns .Models.MyCarAssembly.PaintShop.PaintLine1.Preparing  
-- Preparing is the name of a Conveyor
```

See also

[Open Location \[described\]](#)



on the [Home Ribbon Tab](#)

[LocationInFrame \[SimTalk\]](#)

NumAttr [SimTalk]

Returns the number of user-defined attributes of the object designated by <Path>.

Remarks

The first user-defined attribute has the number 1.

Type

Read-only attribute

Syntax

```
<Path>.NumAttr
```

Return Value

The return value has the data type *integer*.

Is 0 if no user-defined attributes exist.

Example

```
var index: integer
for index := 1 to MyStation.NumAttr
    print MyStation.getAttrName(index)
next
```

See also

[User-defined Attributes \[general description\]](#)

[New \[user-defined attribute\]](#)

NumChildren [SimTalk]

Returns the number of children of the object designated by <Path> and which inherit their settings from it.

Type

Read-only attribute

Syntax

```
<Path>.NumChildren → integer
```

Return Value

The return value has the data type *integer*.

Example

```
print MyStation.NumChildren  
print Transporter.NumChildren
```

SimTalk

[childNo \[SimTalk\]](#)

See also

[Instance \[general description\]](#)

Origin [SimTalk] - general description

Returns the object from which the object designated by <Path> was derived most recently.

Remarks

Compare this example: We copied the object *Frame* three times and named it *Frame1*, *Frame2*, and *Frame3*. Then, we insert *Frame1* into *Frame2*, and *Frame2* into *Frame3*. When we call `.Frame3.Frame2.Frame1.Origin` the result will be **.Frame2.Frame1**, as *.Frame2.Frame1* is the object from which `.Frame3.Frame2.Frame1` was derived most recently.

Type

Read-only attribute

Syntax

```
<Path>.Origin → object
```

Return Value

The return value has the data type *object*.

Example

```
if local_change
    current.openDialog
else
    current.Origin.openDialog
end
```

SimTalk

[OriginRoot \[SimTalk\]](#)

See also

[Show Origin \[class library\]](#)

[Open Origin \[described\]](#)

[Go to Origin \[described\]](#)



on the [Home Ribbon Tab](#)

OriginRoot [SimTalk]

Returns the root object of the inheritance chain if the object designated by <Path> was inherited across several levels.

Type

Read-only attribute

Syntax

```
<Path>.OriginRoot → object
```

Return Value

The return value has the data type *object*.

Example

```
print Station21111111.OriginRoot // might return .MaterialFlow.Station2
```

SimTalk

[Origin \[SimTalk\] - general description](#)

RootFrame [SimTalk]

Returns the root frame of the object designated by <Path>.

Remarks

The root frame is the *Frame* that is located highest up in the hierarchy of *Frames*. The root frame is always located in a folder.

Type

Read-only attribute

Syntax

```
<Path>.RootFrame → object
```

Return Value

The return value has the data type *object*.

VOID if no root frame for the object exists. This applies to all classes that are not of type *Frame*.

Example

```
print MyDialog.RootFrame  
// might return .Models.Model
```


UUID [SimTalk]

Returns the *UUID* (Universally Unique Identifier) of the object designated by <Path> which it keeps during its entire lifetime.

Type

Read-only attribute

Syntax

```
<Path>.UUID → string
```

Return Value

The return value has the data type *string*.

Example

```
print Station.UUID  
// returns 3daf7c80-9411-4671-966f-f3ecd2c0f478 for example
```

See also

[What is a Folder Model?](#)

Attributes of All Objects

Attributes of All Objects

The objects in the *Class Library* have predefined attributes controlling their behavior or representing their state.

- In the dialog windows of the objects you select or enter settings with the check boxes, the text boxes and the drop-down lists on the tabs of the object.
- In *SimTalk* you can assign values to the respective attributes.

For most **attributes** you can set and get their values.

- To **set the value** of an attribute, you might, for example, type:

```
MyStation.Pause := true
```